

a) Object Oriented Programming

Object-Oriented Programming (OOP) is a programming paradigm that organizes data and behavior into reusable units called objects. Here are some key concepts in OOP:

- **Classes:** Classes are blueprints for creating objects. They define the attributes (data) and methods (functions) that objects of that class will have.
- **Objects:** Objects are instances of classes. They encapsulate data and behavior, allowing you to model real-world entities in code.
- **Encapsulation:** Encapsulation is the practice of hiding the internal details of an object and exposing a controlled interface. It helps in data security and maintains a clean interface for interacting with the object.
- **Inheritance:** Inheritance allows one class (subclass or derived class) to inherit properties and methods from another class (superclass or base class). It promotes code reuse and establishes an "is-a" relationship between classes.
- **Polymorphism:** Polymorphism allows objects of different classes to be treated as objects of a common base class. It enables method overriding and dynamic dispatch, improving code flexibility.
- **Abstraction:** Abstraction is the process of simplifying complex reality by modeling classes based on the essential properties and behaviors. It helps in managing complexity and focusing on what's important.

b) Inheritance

Inheritance is a fundamental concept in Object-Oriented Programming (OOP) that allows a new class to inherit properties and behaviors from an existing class. Here are key points about inheritance:

- **Superclass/Base Class:** The class from which properties and methods are inherited is called the superclass or base class.
- **Subclass/Derived Class:** The class that inherits from the superclass is called the subclass or derived class.
- **"Is-a" Relationship:** Inheritance establishes an "is-a" relationship between the superclass and subclass. For example, if you have a superclass `Animal` and a subclass `Dog`, you can say that "Dog is an Animal."
- **Code Reusability:** Inheritance promotes code reuse, as you can create a new class that inherits features from an existing class, reducing duplication.

- **Method Overriding:** Subclasses can override methods inherited from the superclass, providing their own implementation. This allows customization of behavior.

- **Access to Superclass Members:** Subclasses can access public and protected members of the superclass. Private members are not directly accessible.

c) Thread Life Cycle

Thread life cycle refers to the various states a thread can be in during its execution. In Java, threads are objects that execute concurrently within a program. The thread life cycle consists of the following states:

1. **New:** In this state, a thread is created but not yet started. The `Thread` object is instantiated, but the `start()` method hasn't been called.

2. **Runnable:** When the `start()` method is invoked, the thread enters the runnable state. It's ready to run but may not be currently executing due to CPU scheduling.

3. **Running:** In this state, the thread is executing its task and actively using CPU resources.

4. **Blocked/Waiting:** A thread can enter this state when it's waiting for a resource or condition to be met. It's not actively using CPU resources while blocked.

5. **Timed Waiting:** Threads can enter this state when they're waiting for a specific time duration to pass.

6. **Terminated:** When a thread has completed its task or explicitly terminated using the `stop()` method, it enters the terminated state and cannot be restarted.

These states represent the life cycle of a thread in a Java program, and they help manage concurrent execution and synchronization of tasks.

1. Notes on Object Oriented Programming.

☐ Object Oriented Programming

☐ Object-oriented programming (OOP) is a programming language model organized around objects rather than "actions" and data rather than logic.

☐ OOP focuses on what (object) developers want to manipulate rather than how (logic) they want to

manipulate them.

☐ Historically, a program has been viewed as a logical procedure that takes input data, processes it, and produces output data.

☐ Java, C++, C#

2. Notes on WHAT IS JAVA?,

☐ A simple, object-oriented, distributed, interpreted, robust, secure, architecture neutral, portable, high-performance, multithreaded, and dynamic language!

☐ Originally, it was targeted for digital cable television to make it interactive.

☐ Worked fabulous for Internet.

☐ Now it is everywhere.

Originally developed by James Gosling at Sun Microsystems (later acquired by Oracle)

☐ James Gosling, Mike Sheridan, and Patrick Naughton initiated the Java language project in June 1991.

☐ Released in 1995

☐ Java was originally designed for interactive television,

☐ but it was too advanced for the digital cable television industry at the time.

☐ Evolution of Name.

☐ Oak->Green->Java

3. JRE, JDK & JVM.

☐ JRE:

☐ JRE is an acronym for Java Runtime Environment.

☐ It is used to provide runtime environment.

☐ It is the implementation of JVM.

☐ It physically exists. It contains

set of libraries + other files that JVM uses at runtime.

- ☐ JDK

- ☐ JDK is an acronym for Java Development Kit.

- ☐ It physically exists.

- ☐ It contains JRE + development tools

JVM

- ☐ JVM (Java Virtual Machine) is an abstract machine. It is a specification that provides runtime environment in which java bytecode can be executed.

- ☐ JVMs are available for many hardware and software platforms.

- ☐ JVM, JRE and JDK are platform dependent because configuration of each OS differs.

- ☐ But, Java is platform independent.

- ☐ The JVM performs following main tasks:

- ☐ Loads code

- ☐ Verifies code

- ☐ Executes code

- ☐ Provides runtime environment

4. Java DATA TYPES.

Divided into two broad categories:

- ☐ primitive types

- ☐ class/reference types.

- ☐ Primitive data : eight types

- ☐ Logical: boolean (true or false)

- ☐ doesn't hold integer (unlike C)

- ☐ Textual: char (16 bits)

- ☐ use the Unicode(International: 0-255) not ASCII(1 byte: 0-127)

- ☐ Integral: byte (8 bits), short (16 bits), int (32 bits), and long (64 bits)

☐ Floating point: float (32 bits) and double (64 bits)

☐ Class or reference data: two types

☐ Textual: String

☐ All classes that declare by yourself

5. ARRAYS, Major differences with C/C++ arrays.

☐ An array is a collection of data items, all of the same type, accessed using a common name.

☐ The data type can be either a primitive data type or a reference type.

☐ Major differences with C/C++ arrays:

☐ Java arrays are references

☐ Java arrays know their size (length property)

☐ Java multidimensional arrays do not need to be rectangular.

☐ Java array elements are initialized

7. TWO PARADIGMS IN OOP.

☐ All computer programs consist of two elements:

☐ code and data.

☐ A program can be conceptually organized around its code or around its data.

☐ That is, some programs are written around “what is happening” and others are written around “who is being affected.”

☐ These are the two paradigms that govern how an OOP program is constructed.

☐ In Procedural language data and operation/code are separate

☐ Example (using C)

☐ Student – 4 fields: name, id, cgpa, creditCompleted

☐ Update cgpa – need a function to calculate the new cgpa

8. CONSTRUCTOR.

A constructor

☐ Allocate space for instance variables.

☐ initializes an object(its instance variables) immediately upon creation.

☐ Syntax:

☐ It has the same name as the class.

☐ syntactically similar to a method.

☐ Except has no return type. Not even void.

☐ This is because the implicit return type of a class' constructor is the class type itself.

DEFAULT CONSTRUCTOR

☐ When you do not explicitly define a constructor for a class,

☐ then Java creates a default constructor for the class.

☐ This is why the first BankAccount class code worked even though that did not define a constructor.

☐ The default constructor automatically initializes all instance variables to their default values,

☐ Default value

☐ Boolean -> false;

☐ All primitive except boolean -> 0 (zero)

☐ Reference type -> null

☐ Once you define your own constructor, the default constructor is no longer used.

9.SCOPE OF VARIABLE

☐ What is scope?

☐ A scope determines what variable are visible to other parts of your program. Or where the variable is accessible?

☐ It also determines the lifetime of those variable.

☐ A block defines a scope.

☐ the statements between opening and closing curly braces.

☐ As a general rule, variables declared inside a scope are not visible (that is, accessible) to code that is defined outside that scope.

- ☐ Within a block, variables can be declared at any point, but are valid only after they are declared.
- ☐ Thus, if you define a variable at the start of a method, it is available to all of the code within that method.
- ☐ Conversely, if you declare a variable at the end of a block, it is effectively useless, because no code will have access to it.

10. GARBAGE COLLECTION

- ☐ when no references to an object exist, that object is assumed to be no longer needed, and the memory occupied by the object can be reclaimed.
- ☐ Garbage collection only occurs sporadically (if at all) during the execution of your program.
- ☐ Different Java run-time implementations will take varying approaches to garbage collection,
- ☐ but for the most part, you should not have to think about it while writing your programs.

11. TYPES OF INHERITANCE

- ☐ Single inheritance
 - ☐ One parent-> One child
- ☐ Multi level inheritance
 - ☐ one can inherit from a derived class, thereby making this derived class the base class for the new class
- ☐ Hierarchical inheritance
 - ☐ One parent – multiple children
- ☐ Multiple inheritance
 - ☐ Multiple parents -> one child
- ☐ Hybrid inheritance
 - ☐ Is a combination of Single and Multiple inheritance
 - ☐ Java does not support multiple inheritance

INHERITANCE – SUPER KEYWORD

- ☐ The super keyword in java is a reference variable that is

used to refer immediate parent class object.

☐ Whenever you create the instance of subclass, an instance of parent class is created implicitly i.e. referred by super reference variable.

☐ Usage of java super Keyword

☐ super is used to refer immediate parent class member(instance variable & method).

☐ super() is used to invoke immediate parent class constructor.

12. ENCAPSULATION

☐ Encapsulation is the mechanism that binds together code and the data it manipulates and keeps both safe from outside interference and misuse.

☐ One way to think about encapsulation is as a protective wrapper

☐ that prevents the code and data from being arbitrarily accessed by other code defined outside the wrapper.

☐ the basis of encapsulation is the class.

☐ Use access modifier to provide encapsulation

☐ Since the purpose of a class/encapsulation is to encapsulate complexity, there are mechanisms for hiding the complexity of the implementation inside the class.

☐ Use access modifier to specify which members should/ shouldn't be accessed by outside world.

☐ the public interface should be carefully designed not to expose too much of the inner workings of a class

13. METHOD OVERRIDING

- ☐ Means a child class is re-implementing a method of its super class. Or
- ☐ A class replacing an ancestor's implementation of a method with an implementation of its own.
- ☐ When overriding a method in child class
- ☐ Method Signature(name and argument list) and return type must be the same as the parent method.
- ☐ Child method could be equal or more accessible.

METHOD OVERRIDING

- ☐ When an overridden method is called from within its subclass, it will always refer to the version of that method defined by the subclass.
- ☐ The version of the method defined by the superclass will be hidden.
- ☐ To invoke parent method in child class, need to use “super” keyword.
- ☐ Static methods can not be overridden
- ☐ Dynamic binding

14. POLYMORPHISM – BENEFITS

- ☐ Assume the above hierarchy.
- ☐ Consider each class has a draw() method to draw the specific shape.
- ☐ Also consider a method that will take a Shape reference variable (which can hold a shape object or any of its subclass's object) as argument and call the draw method. E.g.


```
static void drawShape(Shape shape){
    shape.draw();
}
```
- ☐ What will happen if you call the method using a Triangle object or a Circle object.

`drawShape(new Triangle()); // will call the draw method of Triangle class`

`drawShape(new Circle()); // will call the draw method of Circle class`

☐ The same invocation can produce “many forms” of results

☐ We can override method in subclasses and which implementation to be used is decided at runtime depending upon the situation (i.e., data type of the real object)

☐ Polymorphism enables programmers to deal with generalities and

☐ let the execution-time environment handle the specifics.

☐ Programmers can command objects to behave in manners appropriate to those objects,

☐ without knowing the types of the objects

☐ (as long as the objects belong to the same inheritance hierarchy).

☐ Now think : - What need to change in the old hierarchy or `drawShap(Shape shape)` method, if you add a new subtype “Square” as shown below.

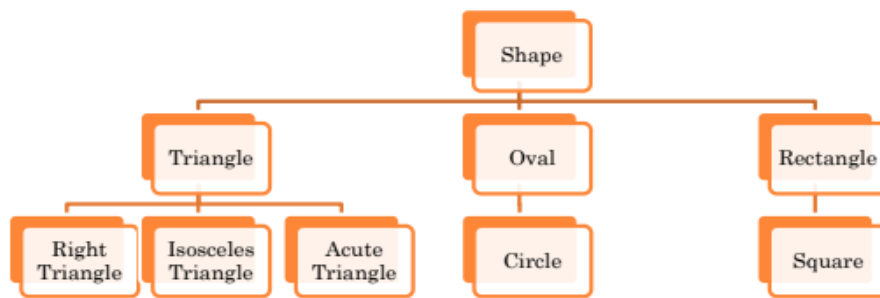
☐ Now think : - What need to change in the old hierarchy or `drawShap(Shape shape)` method, if you add a new subtype “Square” as shown below.

☐ Polymorphism Promotes Extensibility

☐ New object types that can respond to existing method calls can be

☐ incorporated into a system without requiring modification of the base system.

☐ Only client code that instantiates new objects must be modified to accommodate new types.

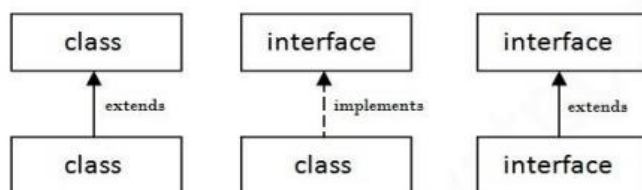


15. Abstraction vs. encapsulation

- Abstraction represent taking out the behavior from how exactly its implemented,
- one example of abstraction in Java is interface
- Encapsulation means hiding details of implementation from outside world so that when things change no body gets affected.
- One example of Encapsulation in Java is private methods; clients don't care about it, You can change, amend or even remove that method if that method is not encapsulated and it were public all your clients would have been affected.
- Encapsulation(hiding complexity) implements of abstraction(show what is only necessary)
- Abstraction is the thought process or model
- Encapsulation is the implementation

16. Relationship between classes & interfaces

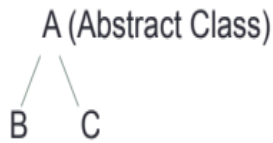
- As shown in the figure given below, a class extends another class, an interface extends another interface but a class implements an interface.



16. When to use - Abstract Class vs. Interface

- If a method in B is so different from the same method in C, there is no shared implementation possible in A.

- We can make the method and A an abstract classes. The methods in A then indicate which methods must be implemented in B and C. A can act as type, which can hold objects of type B or C.



When to use - Abstract Class vs. Interface

- But if all the methods of B must be implemented differently than the same method in C. And there is no common attributes between B and C, make A an interface.



###Lecture 7 to start

a. Java Identifier Rules with Examples

In Java, identifiers are names given to classes, variables, methods, and labels. They follow certain rules:

1. **Must begin with a letter, currency character (\$), or underscore (_)**
 - Valid: variable, \$value, _count
 - Invalid: 123number, @symbol
2. **After the first character, identifiers can also contain digits**
 - Valid: variable123, value_1
 - Invalid: 123variable, 1value
3. **No spaces or special characters are allowed except for currency character (\$) and underscore (_)**
 - Valid: total_amount, user\$ID
 - Invalid: user ID, @username
4. **Identifiers are case-sensitive**
 - variableName and variablename are different identifiers.
5. **Cannot be a Java keyword**
 - Invalid: int, public, class

b. Major Difference between Java Arrays and C/C++ Arrays

The major differences between Java arrays and C/C++ arrays are:

- **Dynamic Sizing:** In Java, arrays are dynamically sized objects. Once an array is created, its size cannot be changed. In C/C++, you can dynamically allocate or resize arrays using functions like `malloc()` or `realloc()`.
- **Bounds Checking:** Java performs automatic bounds checking, preventing the program from accessing elements outside the array's bounds. In C/C++, accessing elements beyond the array's boundary leads to undefined behavior.
- **Objects vs. Pointers:** In Java, arrays are objects, which means they inherit properties and methods from the `Object` class. In C/C++, arrays are essentially pointers to memory locations.
- **Type Safety:** Java arrays are type-safe, meaning they store elements of a specific data type. In C/C++, arrays can hold elements of different data types within the same array, which can lead to type-related issues.

c. Java Garbage Collection with Examples

Java's garbage collection is a process by which Java programs perform automatic memory management. It identifies and deletes objects that are no longer reachable, freeing up memory. Here's an example to illustrate how garbage collection works:

```
class Example {
    public static void main(String[] args) {
        // Creating objects
        MyClass obj1 = new MyClass();
        MyClass obj2 = new MyClass();

        // Assigning obj1 reference to obj2, making obj2 unreachable
        obj2 = obj1;

        // Making obj1 null, making both objects unreachable
        obj1 = null;

        // At this point, both objects are unreachable and eligible for
        // garbage collection
        // Java's garbage collector will automatically reclaim the memory
        // occupied by obj1 and obj2
    }
}

class MyClass {
    // Class definition
}
```

In this example, `obj1` and `obj2` are references to objects of the `MyClass` class.

When `obj2` is assigned the reference of `obj1`, the original reference to the second object is lost, making it unreachable. Setting `obj1` to `null` makes both objects unreachable, and they are eligible for garbage collection. The Java Virtual Machine (JVM) will automatically identify these unreachable objects and release their memory resources.

Write a Java Program to take 5 integer inputs from the user, then sum up only the odd numbers, and then print the sum.

Write a Java Program to take 6 integer inputs from the user, then sum up only the prime numbers, and then print the sum.

1. package Fall2022.Final.Two;

import java.util.Scanner;

```
public class A {  
    public static void main(String[] args) {  
        Scanner Scanner = new Scanner(System.in);  
  
        int[] arr = new int[5];  
        for (int i = 0; i < 5; i++) {  
            arr[i] = Scanner.nextInt();  
        }  
  
        int sum = 0;  
        for (int i = 0; i < 5; i++) {  
            if (arr[i] % 2 != 0) {  
                sum += arr[i];  
            }  
        }  
  
        System.out.println(sum);  
  
        Scanner.close();  
    }  
}
```

2. package Fall2022.Final.Two;

import java.util.Scanner;

```
public class B {  
    public static boolean isPrime(int n) {  
        if (n == 1) return false;
```

```

        for (int i = 2; i <= n / 2; i++) {
            if (n % i == 0) {
                return false;
            }
        }

        return true;
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int[] arr = new int[6];
        for (int i = 0; i < 6; i++) {
            arr[i] = scanner.nextInt();
        }

        int sum = 0;
        for (int i = 0; i < 6; i++) {
            if (isPrime(arr[i])) {
                sum += arr[i];
            }
        }

        System.out.println(sum);

        scanner.close();
    }
}

```

- 2 Write a java program that will take 2 integer inputs from the user and do the following.
- If the first number is even, take 10 inputs from user and sum up only the even numbers and print the result.
 - If 2nd number is greater than the first number, print the first 5 prime numbers.

```
package Spring2022.Final;

import java.util.Scanner;

public class Two {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        int a = scanner.nextInt();
        int b = scanner.nextInt();

        if (a % 2 == 0) {

            int[] arr = new int[10];

            int sum = 0;

            for (int i = 0; i < 10; i++) {

                arr[i] = scanner.nextInt();

                if (arr[i] % 2 == 0) {

                    sum += arr[i];

                }

            }

            System.out.println(sum);

        }

    }

}
```

```
if (b > a) {

    int count = 0;

    int i = 2;

    while (count < 5) {

        boolean isPrime = true;

        for (int j = 2; j < i; j++) {
```



```

        if (i % j == 0) {
            isPrime = false;
            break;
        }
    }

    if (isPrime) {
        System.out.print(i + " ");
        count++;
    }

    i++;
}

}

scanner.close();
}
}

```

Differentiate between abstract classes and interfaces in Java

In Java, abstract classes and interfaces serve different purposes but both are used to achieve abstraction, allowing you to define methods without necessarily providing an implementation.

Abstract Classes:

- An abstract class is a class that cannot be instantiated on its own. It serves as a blueprint for other classes and can contain both abstract methods (methods without implementation) and concrete methods (methods with implementation).
- Abstract classes can have member variables, constructors, and defined methods.
- A class can extend only one abstract class due to Java's single inheritance rule.

Example:

```

public abstract class Shape {
    // Abstract method
    public abstract double calculateArea();

    // Concrete method
    public void display() {

```

```

        System.out.println("This is a shape.");
    }
}

```

Interfaces:

- An interface in Java is a collection of abstract methods. It cannot contain any concrete method implementations.
- It allows a class to implement multiple interfaces, enabling a form of multiple inheritance in Java.
- All methods in an interface are by default public and abstract.

Example:

```

public interface Drawable {
    // Abstract method
    void draw();

    // Another abstract method
    double calculateArea();
}

```

Key Differences:

1. **Implementation:** Abstract classes can have both abstract and concrete methods, while interfaces can only have abstract method declarations.
2. **Inheritance:** A class can extend only one abstract class, but it can implement multiple interfaces.
3. **Fields:** Abstract classes can have fields (member variables) that can be inherited by subclasses. Interfaces cannot contain fields.
4. **Accessibility:** Abstract class methods can have different access modifiers, while interface methods are by default public and abstract.

When to use which?

- Use an abstract class when you have a base class with some default behavior and you want to enforce some methods to be implemented by subclasses.
- Use an interface when you want to define a contract that multiple unrelated classes can implement, ensuring consistency in behavior across these classes.

Both concepts serve the purpose of abstraction in Java, but their usage depends on the specific design and requirements of your program.

Polymorphism is one of the key features of OOP. Explain how polymorphism can be achieved in Java

In Java, polymorphism allows objects of different classes to be treated as objects of a common superclass through inheritance. There are two main types of polymorphism in Java: compile-time polymorphism (achieved through method overloading and method overriding) and runtime polymorphism (achieved through method overriding and interface implementation).

1. **Method Overloading:** This is a form of compile-time polymorphism where multiple methods have the same name but different parameters within the same class. Java determines which method to execute based on the number and types of arguments passed during compile-time.

```
2. class Example {
3.     void display(int num) {
4.         System.out.println("Number: " + num);
5.     }
6.
7.     void display(String text) {
8.         System.out.println("Text: " + text);
9.     }
}
```

Method Overriding: This is a form of runtime polymorphism that occurs when a subclass provides a specific implementation of a method that is already defined in its superclass. It allows a subclass to provide its own implementation of a method defined in its superclass.

```
class Animal {
    void makeSound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("Dog barks");
    }
}
```

Interface Implementation: Java also achieves polymorphism through interfaces, where multiple classes can implement the same interface and provide their own implementation for the interface methods. This allows different objects to be treated uniformly through the interface type.

```
interface Shape {
    void draw();
}

class Circle implements Shape {
    @Override
    public void draw() {
        System.out.println("Drawing Circle");
    }
}

class Rectangle implements Shape {
    @Override
    public void draw() {
        System.out.println("Drawing Rectangle");
    }
}
```

Polymorphism in Java allows for flexibility and extensibility in code by enabling the use of objects of different classes through a common interface or superclass. This flexibility is one of the key principles of Object-Oriented Programming.

```
package Spring2023.Final.Two;

import java.util.Scanner;

public class A {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        String name = scanner.nextLine();

        double height = scanner.nextDouble();

        double weight = scanner.nextDouble();

        double bmi = weight / (height * height);

        System.out.println("Welcome " + name + "!");

        System.out.print("Your BMI is " + bmi + " ");

        if (bmi < 18.5) System.out.println("Underweight");

        else if (bmi >= 18.5 && bmi < 24.9) System.out.println("Normal
Weight");

        else if (bmi >= 25 && bmi < 29.9)
System.out.println("Overweight");

        else if (bmi >= 30 && bmi < 34.9) System.out.println("Obesity I");

        else if (bmi >= 35 && bmi < 39.9) System.out.println("Obesity II");

        else System.out.println("Obesity III (Morbidly Obese)");

        scanner.close();

    }

}
```